BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 3

Roll no:16 PRN No: 12411696 Name of Student: Tarunesh Pawar

Subject Name : Deep Learning

Problem Statement :

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

Step 1: Start the program.

Step 2: Import the required libraries such as TensorFlow, Keras, NumPy, Pandas, Matplotlib, and Scikit-learn.

Step 3: Load the Iris dataset using the load_iris() function from Scikit-learn.

Step 4: Separate the dataset into:

- Input features (Sepal Length, Sepal Width, Petal Length, Petal Width)
- Output labels (Setosa, Versicolor, Virginica)

Step 5: Split the dataset into training and testing sets using an 80:20 ratio.

Step 6: Normalize the input features using StandardScaler to improve training performance and convergence.

Step 7: Define a Sequential Neural Network model consisting of:

- An input layer with 4 input features.

- First hidden layer with 16 neurons and ReLU activation.
- Second hidden layer with 8 neurons and ReLU activation.
- Output layer with 3 neurons and Softmax activation.

Step 8: Compile the model using:

- Adam Optimizer
- Sparse Categorical Crossentropy Loss Function
- Accuracy as the evaluation metric

Step 9: Train the model using forward propagation and backpropagation for a fixed number of epochs.

During each epoch:

- Perform forward propagation to compute the predicted outputs.
- Calculate the loss between predicted and actual outputs.
- Apply backpropagation to compute gradients.
- Update the network weights using the Adam optimizer.

Step 10: Evaluate the trained model on the testing dataset to calculate loss and classification accuracy.

Step 11: Train the model using different learning rates (0.1, 0.01, 0.001) and compare their accuracies.

Step 12: Train the model using different numbers of epochs (10, 30, 50) and compare the resulting accuracies.

Step 13: Plot the following graphs:

- Learning Rate vs Accuracy
- Epochs vs Accuracy
- Training Accuracy vs Validation Accuracy
- Training Loss vs Validation Loss

Step 14: Analyze the performance of the model based on accuracy and loss.

Step 15: Stop the program.

Code :

```
import tensorflow as tf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

iris = load_iris()

X = iris.data
y = iris.target

print(X.shape)
print(y.shape)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

def create_model(lr):

    model = Sequential([
        Dense(16, activation='relu', input_shape=(4,)),
        Dense(8, activation='relu'),
        Dense(3, activation='softmax')
    ])

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

```
learning_rates = [0.1,0.01,0.001]

results=[]

for lr in learning_rates:

    model=create_model(lr)

    model.fit(
        X_train,
        y_train,
        epochs=50,
        batch_size=16,
        verbose=0
    )

    loss,acc=model.evaluate(X_test,y_test,verbose=0)

    results.append([lr,acc])

df=pd.DataFrame(results,
                columns=["Learning Rate","Accuracy"])

print(df)

plt.plot(df["Learning Rate"],
         df["Accuracy"],
         marker="o")

plt.xscale("log")
plt.grid(True)
plt.show()

epochs_list=[10,30,50]

results=[]

for ep in epochs_list:

    model=create_model(0.001)

    history=model.fit(
        X_train,
        y_train,
        validation_split=0.2,
```

```
        epochs=ep,
        batch_size=16,
        verbose=0
    )

    loss,acc=model.evaluate(
        X_test,
        y_test,
        verbose=0
    )

    results.append([ep,acc])

edf=pd.DataFrame(results,
                 columns=["Epochs","Accuracy"])

print(edf)



plt.plot(edf["Epochs"],
         edf["Accuracy"],
         marker="o")

plt.grid(True)
plt.show()

model=create_model(0.001)

history=model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=16
)

plt.plot(history.history["accuracy"])
plt.plot(history.history["val_accuracy"])

plt.legend(["Train","Validation"])

plt.xlabel("Epoch")

plt.ylabel("Accuracy")
```

```
plt.show()

plt.plot(history.history["loss"])
plt.plot(history.history["val_loss"])

plt.legend(["Train","Validation"])

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.show()

loss,accuracy=model.evaluate(
    X_test,
    y_test
)

print("Loss:",loss)

print("Accuracy:",accuracy)
```

Output :

```
import tensorflow as tf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
```

```
iris = load_iris()

X = iris.data
y = iris.target
```

```
print(X.shape)
print(y.shape)
```

```
(150, 4)
(150,)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
def create_model(lr):

    model = Sequential([
        Dense(16, activation='relu', input_shape=(4,)),
        Dense(8, activation='relu'),
        Dense(3, activation='softmax')
    ])

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

```
learning_rates = [0.1,0.01,0.001]

results=[]

for lr in learning_rates:

    model=create_model(lr)

    model.fit(
        X_train,
        y_train,
        epochs=50,
        batch_size=16,
        verbose=0
    )

    loss,acc=model.evaluate(X_test,y_test,verbose=0)

    results.append([lr,acc])

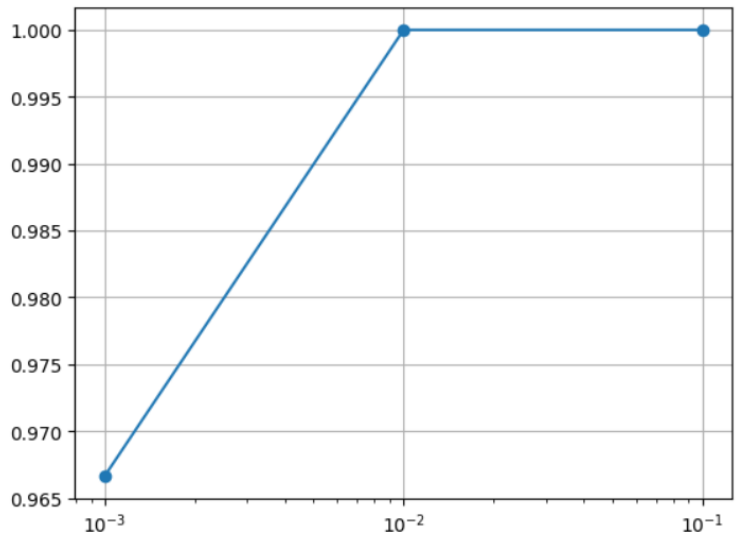
df=pd.DataFrame(results,
                columns=["Learning Rate","Accuracy"])

print(df)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
   Learning Rate  Accuracy
0          0.100  1.000000
1          0.010  1.000000
2          0.001  0.966667
```

```
plt.plot(df["Learning Rate"],
         df["Accuracy"],
         marker="o")

plt.xscale("log")
plt.grid(True)
plt.show()
```



```python
epochs_list=[10,30,50]

results=[]

for ep in epochs_list:

    model=create_model(0.001)

    history=model.fit(
        X_train,
        y_train,
        validation_split=0.2,
        epochs=ep,
        batch_size=16,
        verbose=0
    )

    loss,acc=model.evaluate(
        X_test,
        y_test,
        verbose=0
    )

    results.append([ep,acc])

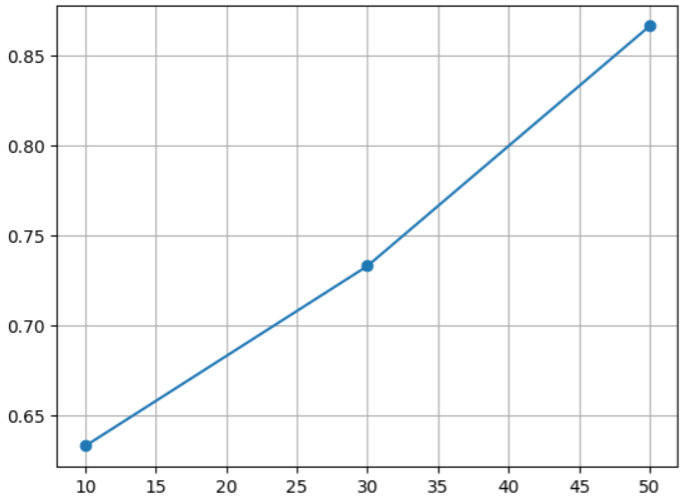
edf=pd.DataFrame(results,
                 columns=["Epochs","Accuracy"])

print(edf)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
   Epochs  Accuracy
0      10  0.633333
1      30  0.733333
2      50  0.866667
```

```python
plt.plot(edf["Epochs"],
         edf["Accuracy"],
         marker="o")

plt.grid(True)
plt.show()
```



```python
model=create_model(0.001)

history=model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=16
)
```

```
Epoch 1/50
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass ar
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
6/6 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.5521 - loss: 1.1460 - val_accuracy: 0.3333 - val_lc
Epoch 2/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.5521 - loss: 1.1133 - val_accuracy: 0.3333 - val_lc
Epoch 3/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.5521 - loss: 1.0836 - val_accuracy: 0.3333 - val_lc
Epoch 4/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5729 - loss: 1.0547 - val_accuracy: 0.3333 - val_lc
Epoch 5/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.5833 - loss: 1.0292 - val_accuracy: 0.3333 - val_lc
Epoch 6/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6042 - loss: 1.0029 - val_accuracy: 0.3333 - val_lc
Epoch 7/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.6042 - loss: 0.9767 - val_accuracy: 0.3333 - val_lc
Epoch 8/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6146 - loss: 0.9507 - val_accuracy: 0.3750 - val_lc
Epoch 9/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.6458 - loss: 0.9232 - val_accuracy: 0.3750 - val_lc
Epoch 10/50
Epoch 11/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6979 - loss: 0.8642 - val_accuracy: 0.5000 - val_l
Epoch 12/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.7396 - loss: 0.8337 - val_accuracy: 0.6250 - val_l
Epoch 13/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step - accuracy: 0.7917 - loss: 0.8031 - val_accuracy: 0.7917 - val_l
Epoch 14/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8021 - loss: 0.7709 - val_accuracy: 0.8750 - val_l
Epoch 15/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8333 - loss: 0.7395 - val_accuracy: 0.8750 - val_l
Epoch 16/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8854 - loss: 0.7064 - val_accuracy: 0.8750 - val_l
Epoch 17/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8542 - loss: 0.6761 - val_accuracy: 0.8750 - val_l
Epoch 18/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8646 - loss: 0.6445 - val_accuracy: 0.8750 - val_l
Epoch 19/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step - accuracy: 0.8646 - loss: 0.6148 - val_accuracy: 0.8750 - val_l
Epoch 20/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8750 - loss: 0.5876 - val_accuracy: 0.9167 - val_l
Epoch 21/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8854 - loss: 0.5600 - val_accuracy: 0.9167 - val_l
Epoch 22/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8854 - loss: 0.5342 - val_accuracy: 0.9167 - val_l
Epoch 23/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8958 - loss: 0.5117 - val_accuracy: 0.9167 - val_l
Epoch 24/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8958 - loss: 0.4903 - val_accuracy: 0.9167 - val_l
Epoch 25/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8854 - loss: 0.4706 - val_accuracy: 0.9167 - val_l
Epoch 26/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8854 - loss: 0.4524 - val_accuracy: 0.9167 - val_l
```

```
Epoch 27/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8854 - loss: 0.4378 - val_accuracy: 0.9167 - val_l
Epoch 28/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8854 - loss: 0.4203 - val_accuracy: 0.9167 - val_l
Epoch 29/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8854 - loss: 0.4066 - val_accuracy: 0.9167 - val_l
Epoch 30/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8958 - loss: 0.3940 - val_accuracy: 0.9167 - val_l
Epoch 31/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8958 - loss: 0.3815 - val_accuracy: 0.9167 - val_l
Epoch 32/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8854 - loss: 0.3707 - val_accuracy: 0.9167 - val_l
Epoch 33/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8854 - loss: 0.3602 - val_accuracy: 0.9167 - val_l
Epoch 34/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8854 - loss: 0.3509 - val_accuracy: 0.9167 - val_l
Epoch 35/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8958 - loss: 0.3420 - val_accuracy: 0.9167 - val_l
Epoch 36/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9062 - loss: 0.3337 - val_accuracy: 0.9167 - val_l
Epoch 37/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9062 - loss: 0.3256 - val_accuracy: 0.9167 - val_l
Epoch 38/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8958 - loss: 0.3179 - val_accuracy: 0.9167 - val_l
Epoch 39/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8958 - loss: 0.3115 - val_accuracy: 0.9167 - val_l
Epoch 40/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9062 - loss: 0.3044 - val_accuracy: 0.9167 - val_l
Epoch 41/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8958 - loss: 0.2979 - val_accuracy: 0.9167 - val_l
Epoch 42/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9062 - loss: 0.2916 - val_accuracy: 0.9167 - val_l
```


```
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.8958 - loss: 0.2979 - val_accuracy: 0.9167 - val_l
Epoch 42/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9062 - loss: 0.2916 - val_accuracy: 0.9167 - val_l
```


```
Epoch 43/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9062 - loss: 0.2861 - val_accuracy: 0.9167 - val_l
Epoch 44/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9167 - loss: 0.2800 - val_accuracy: 0.9167 - val_l
Epoch 45/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9167 - loss: 0.2746 - val_accuracy: 0.9167 - val_l
Epoch 46/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9167 - loss: 0.2694 - val_accuracy: 0.9167 - val_l
Epoch 47/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9167 - loss: 0.2646 - val_accuracy: 0.9167 - val_l
Epoch 48/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9271 - loss: 0.2589 - val_accuracy: 0.9167 - val_l
Epoch 49/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9271 - loss: 0.2547 - val_accuracy: 0.9167 - val_l
Epoch 50/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9375 - loss: 0.2495 - val_accuracy: 0.9167 - val_l
```

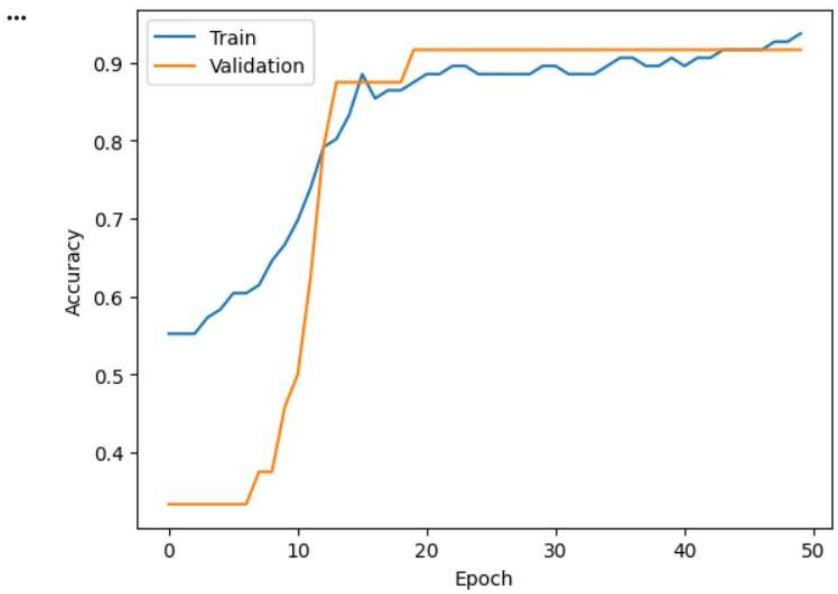
```
plt.plot(history.history["accuracy"])
plt.plot(history.history["val_accuracy"])

plt.legend(["Train","Validation"])

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.show()
```



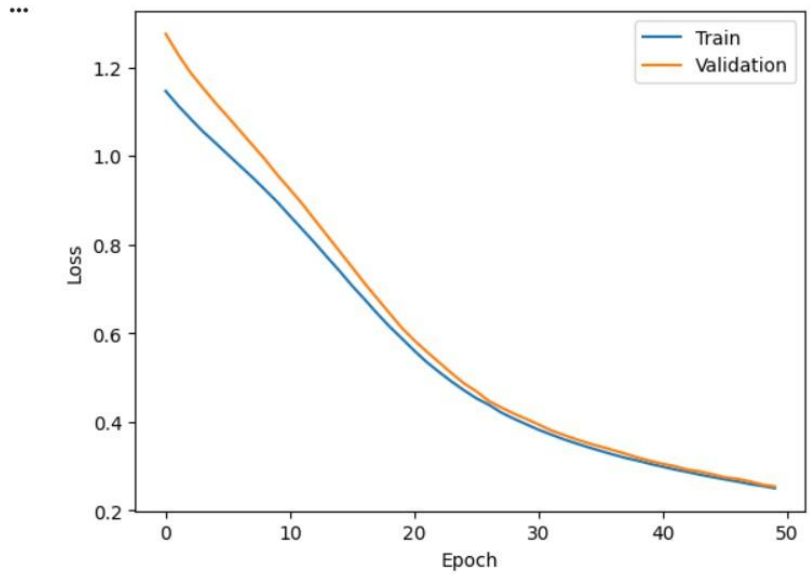

```
plt.plot(history.history["loss"])
plt.plot(history.history["val_loss"])

plt.legend(["Train","Validation"])

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.show()
```



```
loss,accuracy=model.evaluate(
    X_test,
    y_test
)

print("Loss:",loss)

print("Accuracy:",accuracy)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - accuracy: 0.9667 - loss: 0.1840
Loss: 0.18404124677181244
Accuracy: 0.9666666388511658
```